Travaux Pratiques : Regression Lineaire, Regression Logistique

1-La régression linéaire

Les bibliothèques requises

Tout d'abord, nous allons importer les bibliothèques requises

In [1]:

```
import numpy as np

import matplotlib.pyplot as plt

import pandas as pd

import seaborn as sns

from sklearn.datasets import load_boston

%matplotlib inline
```

Chargement de données CSV

On utilise le jeu de données Auto MPG classique et construit un modèle permettant de prédire la consommation de carburant des automobiles de la fin des années 70 et du début des années 80. Pour ce faire, nous fournirons au modèle une description de nombreuses automobiles de cette période. Cette description inclut des attributs tels que: cylindres, déplacement, puissance et poids.

In [2]:

```
dataset_path=" datasets/auto-mpg.csv"

column_names =
['MPG','Cylinders','Displacement','Horsepower','Weight','Acceleration', 'Model
Year', 'Origin']

raw_dataset = pd.read_csv(dataset_path, names=column_names,na_values = "?",
comment='\t',sep=" ", skipinitialspace=True)
```

Nous allons maintenant charger les données dans un cadre de données pandas en utilisant pd.read_csv. Nous imprimons ensuite les 5 premières lignes des données en utilisant head(). Puis faire une copie du dataset

In [3]:

```
raw_dataset.head()
```

Out[3]:

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin
0	18.0	8	307.0	130.0	3504.0	12.0	70	1
1	15.0	8	350.0	165.0	3693.0	11.5	70	1
2	18.0	8	318.0	150.0	3436.0	11.0	70	1
3	16.0	8	304.0	150.0	3433.0	12.0	70	1
4	17.0	8	302.0	140.0	3449.0	10.5	70	1

In [4]:

```
dataSet = raw_dataset.copy()
```

In [5]:

```
dataSet.columns
```

Out[5]:

```
Index(['MPG', 'Cylinders', 'Displacement', 'Horsepower', 'Weight',
       'Acceleration', 'Model Year', 'Origin'],
      dtype='object')
```

In [6]:

```
dataSet.shape
```

Out[6]:

```
(398, 8)
```

Prétraitement des données

Une fois les données chargées, il est recommandé de vérifier s'il existe des valeurs manquantes dans les données. Nous comptons le nombre de valeurs manquantes pour chaque fonctionnalité à l'aide deisnull()

In [7]:

```
dataSet.isnull ().sum ()
```

Out[7]:

```
MPG             0
```

```
Cylinders       0
Displacement    0
Horsepower      6
Weight          0
Acceleration    0
Model Year      0
Origin          0
dtype: int64
```

Il manque 6 valeurs dans la colonne Horsepower, il faut supprimer ces lignes

In [8]:

```
dataSet = dataSet.dropna()
```

In [9]:

```
dataSet.isnull().sum ()
```

Out[9]:

```
MPG             0
Cylinders       0
Displacement    0
Horsepower      0
Weight          0
Acceleration    0
Model Year      0
Origin          0
dtype: int64
```

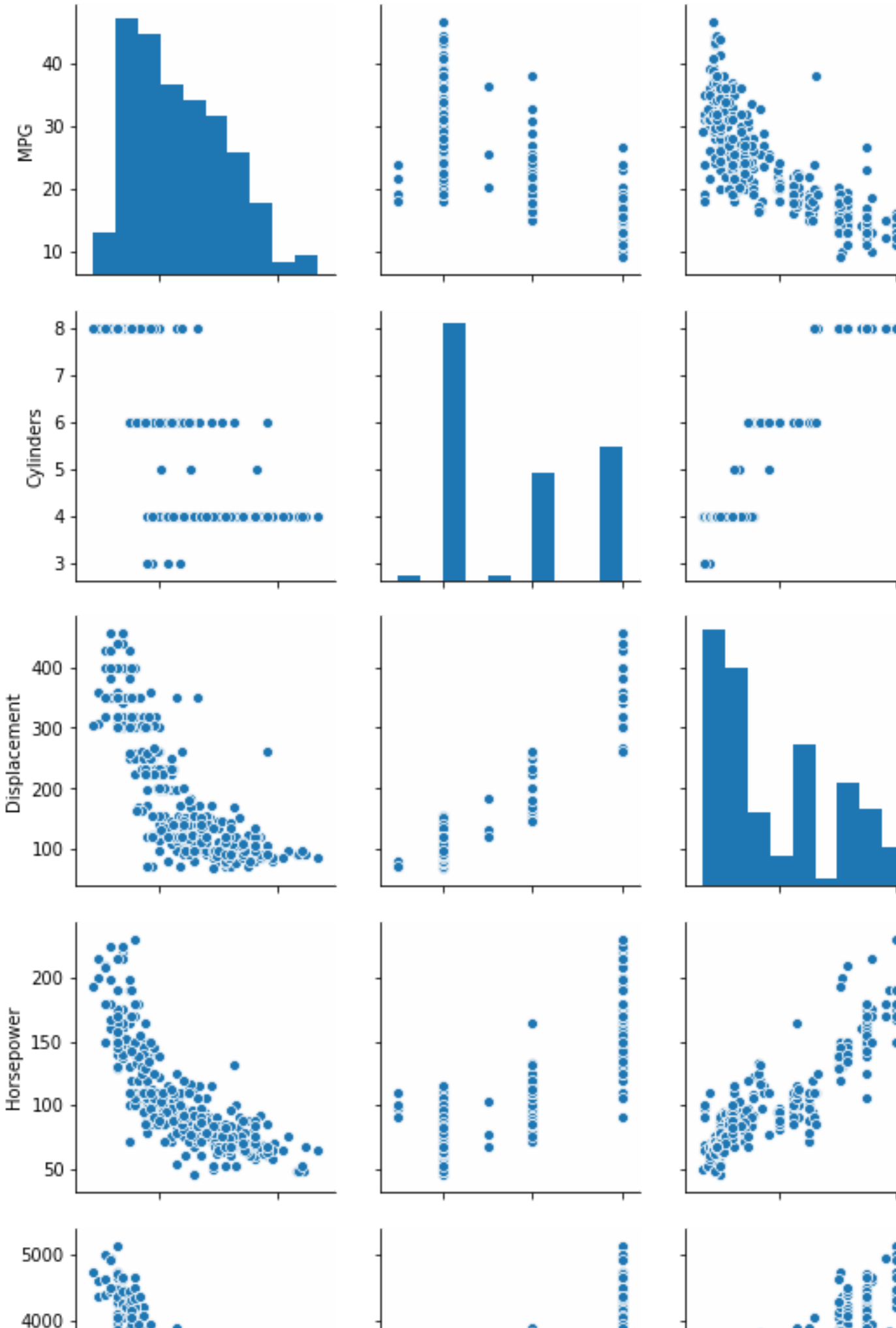
Toutefois, il ne manque aucune valeur dans cet ensemble de données

In [10]:

```
%matplotlib inline
sns.pairplot(dataSet)
```

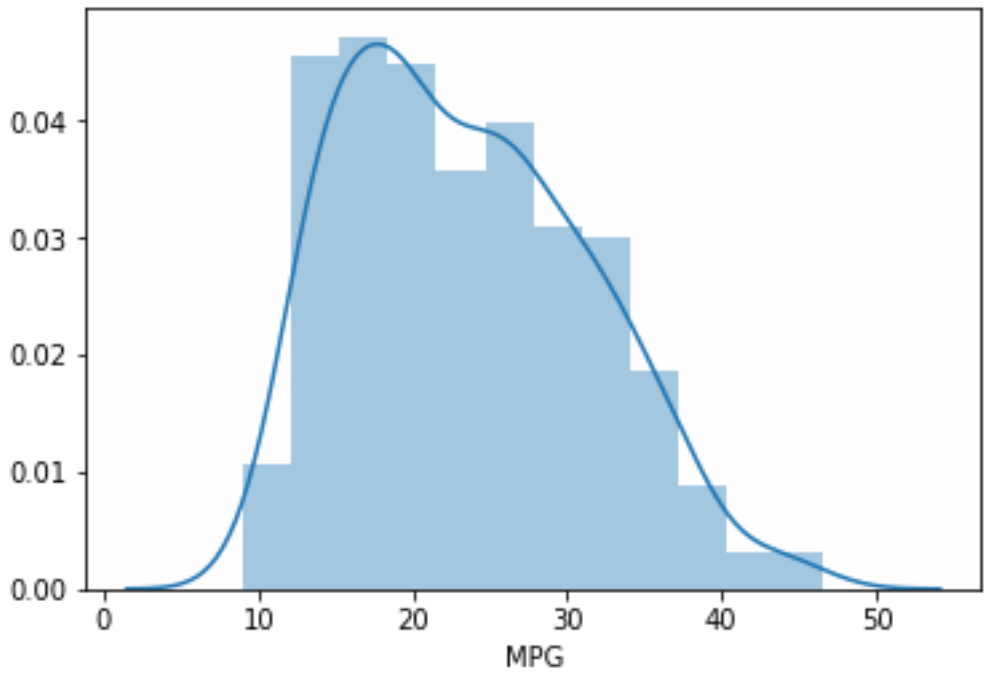
Out[10]:

```
<seaborn.axisgrid.PairGrid at 0x2371dca5978>
```


MPG
Cylinders
Displacement
Horsepower

In [11]:

```
%matplotlib inline
sns.distplot(dataSet['MPG']);
```



Nous voyons que les valeurs de MPG sont distribuées normalement avec peu de valeurs aberrantes.

Ensuite, nous créons une matrice de corrélation qui mesure les relations linéaires entre les variables. La matrice de corrélation peut être formée en utilisant la corrfonction de la bibliothèque de bases de données pandas. Nous utiliserons la heatmapfonction de la bibliothèque Seaborn pour tracer la matrice de corrélation.

In [12]:

```
cor=dataSet.corr()
cor
```

Out[12]:

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin
MPG	1.000000	-0.777618	-0.805127	-0.778427	-0.832244	0.423329	0.580541	0.565209
Cylinders	-0.777618	1.000000	0.950823	0.842983	0.897527	-0.504683	-0.345647	-0.568932
Displaceme	-	0.950823	1.000000	0.897257	0.93299	-0.543800	-	-

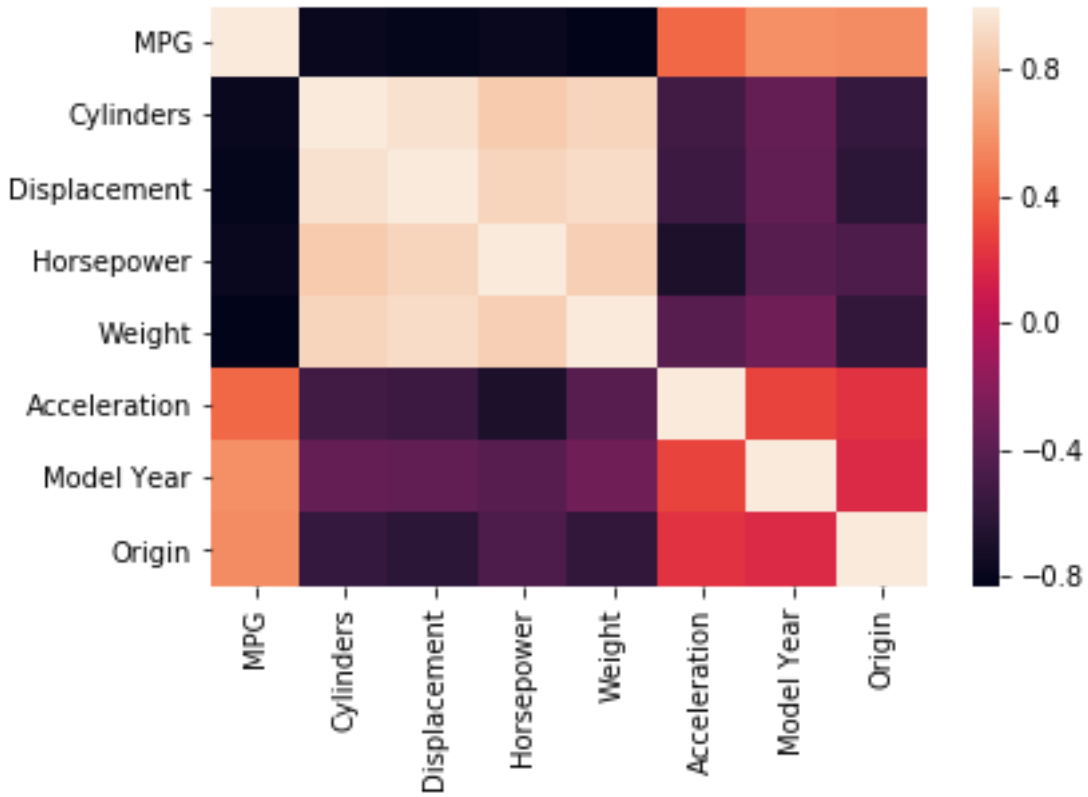
nt	0.805127				4		0.369855	0.614535
Horsepower	-0.778427	0.842983	0.897257	1.000000	0.864538	-0.689196	-0.416361	-0.455171
Weight	-0.832244	0.897527	0.932994	0.864538	1.000000	-0.416839	-0.309120	-0.585005
Acceleration	0.423329	-0.504683	-0.543800	-0.689196	-0.416839	1.000000	0.290316	0.212746
Model Year	0.580541	-0.345647	-0.369855	-0.416361	-0.309120	0.290316	1.000000	0.181528
Origin	0.565209	-0.568932	-0.614535	-0.455171	-0.585005	0.212746	0.181528	1.000000

In [13]:

```
sns.heatmap(cor)
```

Out[13]:

```
<matplotlib.axes._subplots.AxesSubplot at 0x237215f9c50>
```



Le coefficient de corrélation varie de -1 à 1. Si la valeur est proche de 1, cela signifie qu'il existe une forte corrélation positive entre les deux variables. Quand il est proche de -1, les variables ont une forte corrélation négative.

Pour s'adapter à un modèle de régression linéaire, nous sélectionnons les entités présentant une forte corrélation avec notre variable cible MPG. (Displacement,Horsepower,Weight)

Fractionnement des données en ensembles de traitement et de test

Ensuite, nous avons divisé les données en ensembles de formation et de test. Nous formons le modèle avec 70% des échantillons et testons avec les 30% restants.

Nous faisons cela pour évaluer les performances du modèle sur des données invisibles .

Pour fractionner les données, nous utilisons la train_test_splitfonction fournie par la bibliothèque scikit-learn. Nous imprimons enfin les tailles de notre ensemble d'entraînement et de test pour vérifier si le fractionnement a bien eu lieu.

In [14]:

```
X=dataSet.drop('MPG', axis=1)

Y = dataSet['MPG']
```

In [15]:

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.3,
random_state=20)
```

Génération du modèle

In [16]:

```
from sklearn.linear_model import LinearRegression

lin_model = LinearRegression ()

lin_model.fit (X_train,Y_train)
```

Out[16]:

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,

         normalize=False)
```

Évaluation du modèle

In [17]:

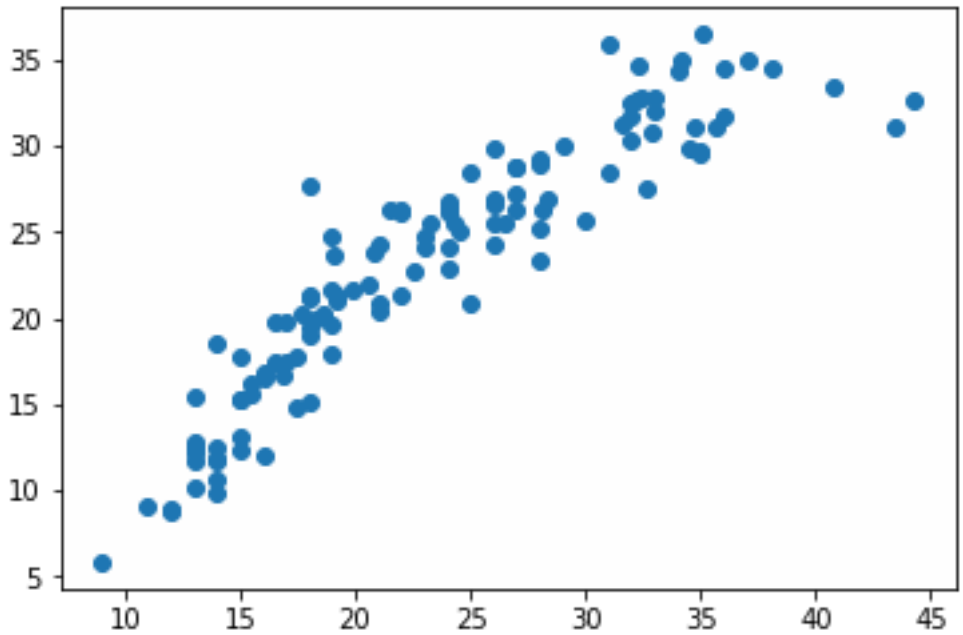
```
predictions=lin_model.predict(X_test)
```

In [18]:

```
plt.scatter(Y_test,predictions)
```

Out[18]:

```
<matplotlib.collections.PathCollection at 0x237218d24e0>
```



In [19]:

```
from sklearn.metrics import  mean_squared_error, r2_score
count_misclassified = (Y_test != predictions).sum()
print('Échantillons mal classés: {}'.format(count_misclassified))
accuracy = r2_score(Y_test, predictions)
print('Précision: {:.2f}'.format(accuracy))
```

```
Échantillons mal classés: 118
Précision: 0.84
```

2-La régression logistique

Les bibliothèques requises

Tout d'abord, nous allons importer les bibliothèques requises

In [20]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import preprocessing
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import metrics
```

Charger le jeu de données

J'ai utiliser les données de régression logit fournies par UCLA (Institut de recherche et d'éducation numériques).

In [21]:

```
dataset = pd.read_csv('datasets/binary.csv')
```

In [22]:

```
dataset.columns = ["admit", "gre", "gpa", "prestige"]
```

admit: l'etudiant admit ou non

gre:Le Graduate Record Examination (GRE) est un test en anglais créé et géré par l'entreprise ETS. Ce test est requis pour étudier dans la plupart des universités ou graduate schools dans des pays anglophones.

gpa:C'est un moyen standard de mesurer les résultats scolaires aux États-Unis.

prestige:niveaux d'études

In [23]:

```
dataset.shape
```

Out[23]:

```
(400, 4)
```

In [24]:

```
dataset.head()
```

Out[24]:

	admit	gre	gpa	prestige
0	0	380	3.61	3
1	1	660	3.67	3
2	1	800	4.00	1
3	1	640	3.19	4
4	0	520	2.93	4

Représentons toutes les colonnes du jeu de données.

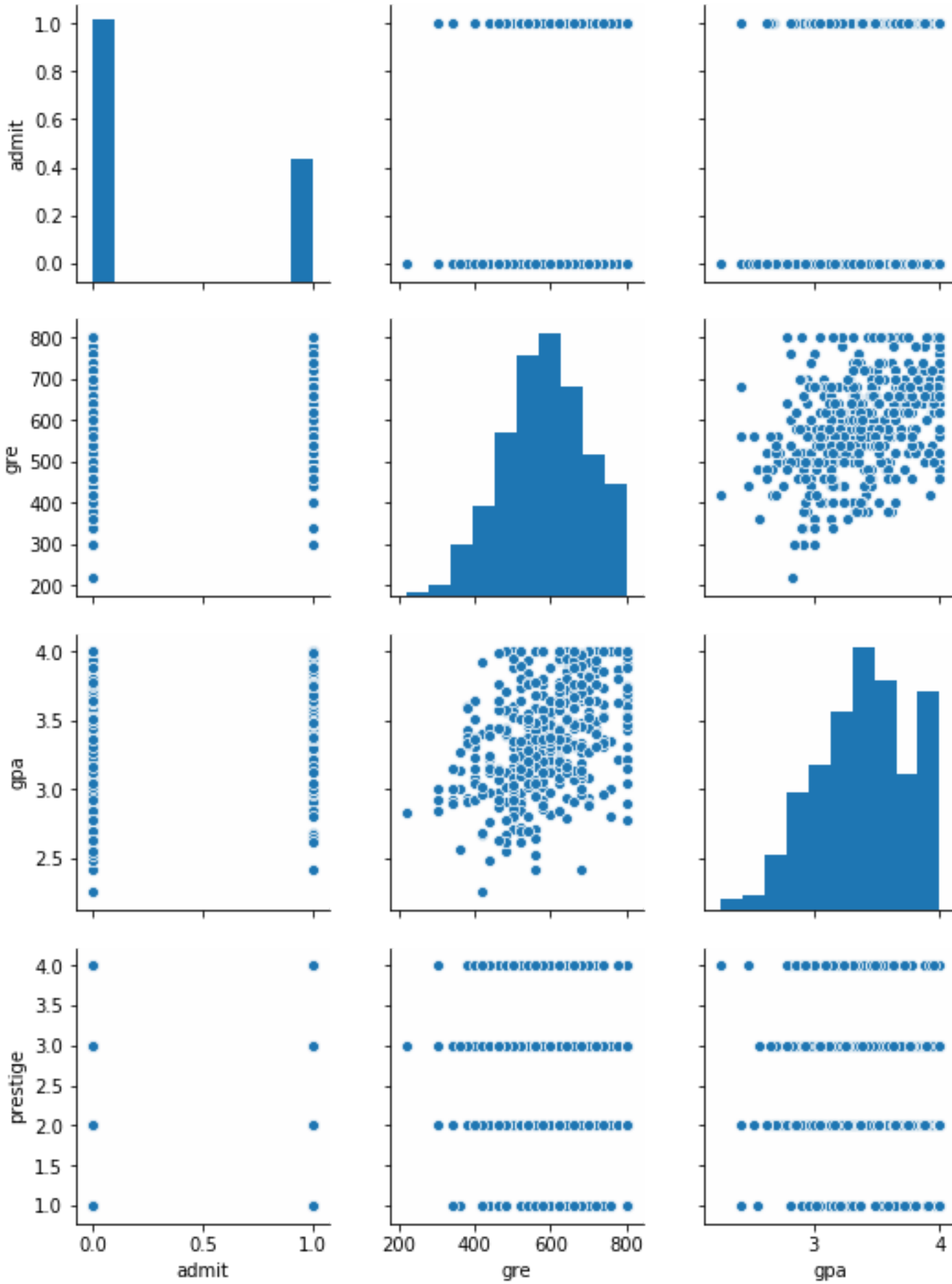
In [25]:

```
%matplotlib inline

sns.pairplot(dataset)
```

Out[25]:

```
<seaborn.axisgrid.PairGrid at 0x2372191e978>
```


admit
gre
gpa
prestige
1.0
0.8
0.6
0.4
0.2
0.0
800
700
600
500
400
300
200
4.0
3.5
3.0
2.5
2.0
1.5
0.5
3
4

Prétraitement des données

Une fois les données chargées, il est recommandé de vérifier s'il existe des valeurs manquantes dans les données. Nous comptons le nombre de valeurs manquantes pour chaque fonctionnalité à l'aide deisnull()

In [26]:

```
dataset.isnull ().sum ()
```

Out[26]:

```
admit       0
gre         0
gpa         0
prestige    0
dtype: int64
```

il ne manque aucune valeur dans cet ensemble de données

La bibliothèque Python pandas offre une grande flexibilité dans la manière dont les variables catégorielles sont représentées.

In [27]:

```
dummy_ranks = pd.get_dummies(dataset['prestige'], prefix='prestige')
```

In [28]:

```
dummy_ranks.head()
```

Out[28]:

	prestige_1	prestige_2	prestige_3	prestige_4
0	0	0	1	0
1	0	0	1	0
2	1	0	0	0
3	0	0	0	1
4	0	0	0	1

Création d'une data prés à la régression

In [29]:

```
a=['admit', 'gre', 'gpa']
data = dataset[a].join(dummy_ranks.iloc[:,:])
```

In [30]:

```
data['admit'].value_counts()
```

Out[30]:

```
0    273
1    127
Name: admit, dtype: int64
```

Fractionnement des données en ensembles de traitement et de test

Ensuite, nous avons divisé les données en ensembles de formation et de test. Nous formons le modèle avec 70% des échantillons et testons avec les 30% restants.

Nous faisons cela pour évaluer les performances du modèle sur des données invisibles .

Pour fractionner les données, nous utilisons la train_test_splitfonction fournie par la bibliothèque scikit-learn. Nous imprimons enfin les tailles de notre ensemble d'entraînement et de test pour vérifier si le fractionnement a bien eu lieu.

In [31]:

```
x = data.drop('admit', axis=1)
y = data['admit']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3,
random_state=1)
```

Nous allons maintenant procéder à une régression logistique, ce qui est assez simple. Nous spécifions simplement la colonne contenant la variable que nous essayons de prédire, suivie des colonnes que le modèle devrait utiliser pour effectuer la prédiction.

Nous prévoyons maintenant la colonne d'admission en fonction des variables nominales gre, gpa et prestige_1, prestige_2, prestige_3 et prestige_4.

Génération du modèle

In [32]:

```
logmodel = LogisticRegression(C=1e20)
logmodel.fit(x_train, y_train)
```

Out[32]:

```
LogisticRegression(C=1e+20, class_weight=None, dual=False, fit_intercept=True,
          intercept_scaling=1, max_iter=100, multi_class='warn',
          n_jobs=None, penalty='l2', random_state=None, solver='warn',
          tol=0.0001, verbose=0, warm_start=False)
```

Modèle obtenu

In [33]:

```
logmodel.intercept_, logmodel.coef_
```

Out[33]:

```
(array([-3.7886289]),
 array([[ 1.52590242e-03,  8.92678921e-01, -2.69476045e-01,
         -5.64976333e-01, -1.33513537e+00, -1.61904115e+00]]))
```

Ainsi notre fonction score est définit comme suit :

?(?)=-3.7886289+ 1.52590242e-03∗?1+8.92678921e-01∗?2+--2.69476045e-01∗?3+-5.64976333e-01∗?4+-1.33513537e+00∗?5+--1.61904115e+00∗?6

Évaluation du modèle

In [36]:

```
y_pred = logmodel.predict(x_test)
y_pred
```

Out[36]:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0,
       1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
       0, 1, 0, 0, 0, 1, 0, 0, 0, 1], dtype=int64)
```

In [37]:

```
count_misclassified = (y_test != y_pred).sum()
print('Échantillons mal classés: {}'.format(count_misclassified))
accuracy = metrics.accuracy_score(y_test, y_pred)
print('Précision: {:.2f}'.format(accuracy))
```

```
Échantillons mal classés: 31
Précision: 0.74
```